Liquid Latent Synthesis (LLS-4): A Quad-Partite Architecture for Latent-Space Concept Interpolation in Continual Reinforcement Learning

Architecture Proposal, Prototype Implementation, and a Preliminary Pilot Study

Rudra Madhab Mishra[1]

[1]Department of Computer Science and Engineering (AI & ML), GIET University, Gunupur, Rayagada, Odisha, India , rudramdhabmishra1@gmail.com

Abstract

Catastrophic forgetting remains a central obstacle to deploying continually-learning agents on resource-constrained edge hardware. Existing mitigations – regularization-based methods such as Elastic Weight Consolidation (EWC) and architecture-growth methods such as Progressive Neural Networks – either accumulate numerically unstable penalty terms as the number of tasks grows, or scale parameter count linearly with the number of skills learned. We propose **Liquid Latent Synthesis (LLS-4)**, an architecture that decouples fast online policy learning from offline concept consolidation. A Closed-form Continuous-time (CfC) liquid network encodes physical experience into a compact 16-dimensional latent manifold during an "awake" phase; during an offline "sleep" phase, a synthesis engine identifies pairs of latent memories that are temporally proximate but semantically distinct, linearly interpolates them, decodes the interpolated vector into synthetic training data, and distills the result into a lightweight (tens-of-thousands-of-parameter) adapter module routed at inference time by a soft-attention context gate. We implement the full pipeline in PyTorch and `ncps`, and report a first pilot comparison against a standard LSTM and a custom diagonal-Fisher EWC baseline on a synthetic two-task regression proxy. The pilot run shows that the current prototype's shared backbone is *not* protected during sequential training, producing a retention score (4.5%) below both baselines (33.5% and 49.2%) – the opposite of the architecture's design intent. A secondary "zero-shot synthesis" metric (71.0%) is also reported, but we identify and disclose a degeneracy in its construction that likely inflates this number independent of any genuine skill combination. We treat both findings as diagnostic rather than confirmatory, and use them to motivate a concrete, itemized experimental protocol – backbone freezing, parameter-matched baselines, multi-seed evaluation, a sleep-phase ablation, and a non-degenerate zero-shot task – required before any claim of solving catastrophic forgetting can be supported. This paper's contribution is therefore the architecture, an open, working prototype, and a transparent account of what remains to be validated.

Keywords: continual learning, catastrophic forgetting, liquid neural networks, closed-form continuous-time networks, latent space interpolation, edge AI, neural adapters

1 Introduction

Sequential learning in neural networks is undermined by *catastrophic forgetting*: when a network trained on task A is subsequently trained on task B, gradient updates that improve performance on

B typically overwrite the parameters responsible for A, and performance on A collapses [1]. This problem is especially acute for autonomous edge systems – robots, drones, and embedded controllers – that cannot rely on cloud retraining and must adapt locally with limited memory and compute.

Two families of methods dominate current practice. *Regularization-based* approaches such as Elastic Weight Consolidation (EWC) estimate, via the diagonal of the Fisher information matrix, which weights were important for previous tasks, and penalize movement away from those weights when learning a new task [2]. Synaptic Intelligence computes a similar importance estimate online rather than post-hoc [4]. These methods are memory-cheap but the penalty terms accumulate across tasks, and as more tasks are learned the network's remaining plastic capacity shrinks, eventually causing instability. *Architecture-growth* approaches such as Progressive Neural Networks instead allocate a new column of parameters per task and use lateral connections to transfer knowledge [5], guaranteeing no forgetting at the cost of parameter count that grows linearly (or worse) with the number of tasks. *Replay-based* approaches – storing raw exemplars, or training a generative model to produce pseudo-samples of old tasks [6], or replaying compressed feature-space exemplars rather than raw inputs [7, 8] – avoid weight-level interference but require either a data buffer or an auxiliary generative model, both of which add memory and compute overhead that is difficult to justify on constrained edge hardware.

Separately, *liquid* and closed-form continuous-time neural networks have been proposed as compact, interpretable controllers for real-time control tasks. Neural Circuit Policies demonstrate that a controller with on the order of tens of neurons and a few hundred synapses can perform end-to-end steering control with strong generalization [9], and Closed-form Continuous-time networks (CfC) make such continuous-time dynamics tractable to train without a numerical ODE solver [10]. These properties – small parameter count, continuous-time operation, strong inductive bias for control – make liquid networks an attractive base learner for edge deployment, independent of the continual-learning question.

This paper's position. We ask whether a small liquid controller can be paired with a mechanism that consolidates new skills *without* touching the controller's existing weights and *without* storing raw exemplars, by instead treating the controller's own compressed latent memory as the substrate for skill synthesis. We propose *Liquid Latent Synthesis* (LLS-4), which borrows the "interpolate, then train" idea used in *mixup* for input-space data augmentation [11] but applies it in a compact 16-dimensional experience-latent space across pairs of past memories, rather than across raw input samples. The interpolated concept is decoded into synthetic training data and distilled into a small, independent adapter network, which is then routed at inference time via soft attention – conceptually a lightweight, generative variant of parameter-isolation methods such as Progressive Neural Networks, but with adapters synthesized automatically from experience rather than allocated per labeled task.

Contributions.

1. A quad-partite architecture specification – environment, working memory, sleep engine, context gate – with explicit scoring, interpolation, and routing equations (Section 3).

2. A full, working PyTorch/`ncps` prototype implementing the pipeline end to end, including an interactive dashboard for inspecting the latent memory manifold and the synthesis process (Section 3, Appendix).

3. A pilot benchmark against an LSTM and a custom EWC baseline on a synthetic two-task regression proxy (Section 4).

4. A transparent report of what the pilot results do and do not demonstrate, including a previously undisclosed degeneracy in the zero-shot metric, and a concrete, itemized protocol for a

properly controlled follow-up study (Sections 5–7).

2 Related Work

Catastrophic forgetting. The phenomenon was first characterized by McCloskey and Cohen in connectionist models [1]. EWC [2] remains the most widely cited regularization-based mitigation; Huszár's subsequent analysis notes that EWC's multi-task penalty accumulation is not fully justified by the Bayesian derivation used to motivate it [3], a concern consistent with the numerical-instability limitation this paper's project notes attribute to EWC at scale.

Architecture isolation and rehearsal. Progressive Neural Networks [5] avoid forgetting entirely by freezing old columns and adding new ones, at linear parameter cost. Deep generative replay [6] and latent/feature-space replay [7, 8] instead rehearse compressed representations of old data alongside new data. LLS-4's context-gated adapters are closest in spirit to parameter isolation (old computation is never overwritten), while its sleep-phase synthesis procedure is closest in spirit to generative/latent replay (new training signal is manufactured rather than stored) – the architecture is best understood as a combination of existing mechanisms applied at the level of a compact policy latent, rather than a fundamentally new learning rule, and its value proposition rests on whether that combination is empirically effective, which is precisely what Sections 4–6 interrogate.

Continuous-time and liquid networks. Neural Circuit Policies [9] and Closed-form Continuous-time networks [10] motivate the use of a CfC backbone as the "fast" online learner in this work, on the basis of their demonstrated parameter efficiency for control tasks.

Latent interpolation as data augmentation. *mixup* [11] showed that linearly interpolating pairs of training examples (and their labels) acts as an effective regularizer in supervised learning. LLS-4's dream-generation step is structurally analogous, but the interpolation is performed on compressed *experience* vectors rather than raw inputs, and the resulting synthetic data is used to train a new, isolated module rather than to regularize the existing one.

3 Proposed Architecture

LLS-4 consists of four decoupled modules, summarized in Figure 1.

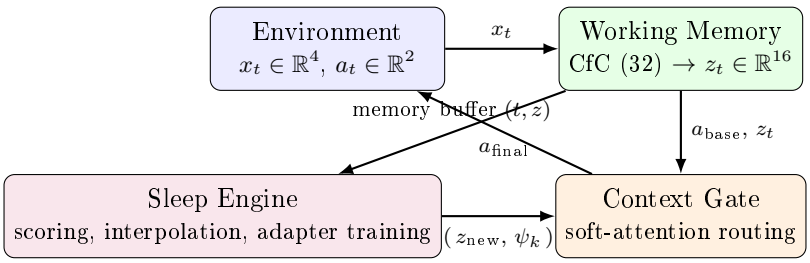

Figure 1: LLS-4 quad-partite architecture. The environment and working memory operate continuously during the "awake" phase; the sleep engine operates offline and injects new adapters into the context gate, which subsequently blends adapter outputs with the base policy at inference time.

3.1 Environment

The prototype environment is a continuous 20×20 2D world. The state is $x_t = [\, d_{\text{target}},\, d_{\text{obstacle}},\, e,\, v \,] \in \mathbb{R}^4$ (distance to target, distance to obstacle, remaining energy, current speed), and the action is

$a_t = [\,\Delta\theta,\, F\,] \in \mathbb{R}^2$ (steering delta, acceleration force). Reaching the target yields +10 reward and relocates the target; contacting the obstacle yields −5 reward and damps velocity; a small per-step time penalty and energy drain proportional to speed complete the dynamics.

3.2 Working Memory

The working memory is a single-layer CfC network [10] with hidden size 32, implemented via the `ncps` library:

$$h_t = \mathrm{CfC}(x_t, h_{t-1}), \tag{1}$$

$$z_t = W_z h_t + b_z, \qquad z_t \in \mathbb{R}^{16}, \tag{2}$$

$$a_{\text{base}} = W_a z_t + b_a, \qquad a_{\text{base}} \in \mathbb{R}^2. \tag{3}$$

Every visited state is compressed into the 16-dimensional latent z_t and stored, together with a timestamp, in a fixed-capacity memory buffer. A separate decoder $D_\phi : \mathbb{R}^{16} \to \mathbb{R}^{4+2}$ ($16 \to 32 \to$ ReLU $\to 6$) reconstructs a state-action pair from a latent vector; this decoder is trained jointly with an auxiliary reconstruction loss and is used only during the sleep phase.

3.3 Sleep Engine

Given a memory buffer of (t_i, z_i) pairs, the sleep engine scores every pair by combining semantic distinctness with temporal proximity:

$$S_{ij} = \big(1 - \cos(z_i, z_j)\big) \;+\; \lambda \cdot P(t_i, t_j), \qquad P(t_i, t_j) = 1 - \frac{|t_i - t_j|}{\max_{k,l} |t_k - t_l|}, \tag{4}$$

with $\lambda = 0.1$, and selects $(i^*, j^*) = \arg\max_{i \neq j} S_{ij}$. This rewards pairs of memories that occurred close together in time but represent *dissimilar* latent concepts.[1] The selected pair is interpolated,

$$z_{\text{new}} = \alpha z_{i^*} + (1 - \alpha) z_{j^*}, \qquad \alpha = 0.5, \tag{5}$$

decoded via D_ϕ into a synthetic state-action pair $(\hat{x}, \hat{a})$, and expanded into a synthetic dataset of N=200 points by adding independent Gaussian noise, $\hat{x} + \epsilon_x$ with $\epsilon_x \sim \mathcal{N}(0, 0.1^2 I_4)$ and $\hat{a} + \epsilon_a$ with $\epsilon_a \sim \mathcal{N}(0, 0.05^2 I_2)$. A new adapter ψ_k – a two-layer MLP, $4 \to 16 \to$ ReLU $\to 2$, under 200 parameters – is trained on this synthetic dataset by MSE regression (Adam, lr $= 0.01$, 50 epochs in the standalone sleep cycle; a faster 30-epoch/100-step variant is used in the interactive dashboard for responsiveness).

3.4 Context Gate

Each synthesized adapter ψ_k is stored together with its originating key $z_{\text{new},k}$. At inference time, given a live query $q = z_t$, the gate computes soft-attention weights over all M stored keys and blends adapter outputs with the base policy:

$$\alpha_k = \frac{\exp(q^\top z_{\text{new},k})}{\sum_{l=1}^{M} \exp(q^\top z_{\text{new},l})}, \qquad a_{\text{final}} = a_{\text{base}} + \sum_{k=1}^{M} \alpha_k \, \psi_k(x_t). \tag{6}$$

[1] We note that this scoring rule, as implemented, differs from a superficially similar formula in an earlier project brief that combined *raw* cosine similarity (rather than $1 - \cos$) with the temporal term; the raw-similarity version would instead favor pairs of *redundant* memories, which is inconsistent with the module's stated intent. Equation 4 is the version we analyze, as it is the version implemented and executed.

Because the adapters are added to, rather than substituted for, the base output, and because the base network's own weights receive no special protection in the current implementation, Equation 6 guarantees that adapters cannot *directly* corrupt the base policy's parameters – but, as Section 5 shows, it does not by itself guarantee that the base policy retains old skills, since the base network continues to be trained on new data unprotected.

4 Experimental Methodology

Baselines. We compare against (i) a standard single-layer LSTM (hidden size 32) with a linear output head, and (ii) a 2-layer MLP (hidden size 64) trained with a custom EWC penalty, using a diagonal empirical-Fisher approximation computed from squared per-sample gradients on Task A data and a quadratic penalty with $\lambda_{\text{EWC}} = 100$.

Task design. Rather than the full navigation environment described in Section 3, this pilot uses a synthetic, controlled two-task regression proxy so that ground-truth targets are known exactly: Task A maps $x \in [0, 10]^4$ to $y_A = [\sin(x_0), \cos(x_1)]$, and Task B maps the same input distribution to $y_B = [-\sin(x_0), -\cos(x_1)]$. *We flag explicitly that this proxy task was not run inside* `environment.py`; evaluating the architecture on the navigation-and-avoidance domain it was designed for is left to future work (Section 7).

Procedure. All three models are trained on Task A (100 gradient steps), evaluated, then trained sequentially on Task B (100 steps; EWC additionally applies its Fisher-weighted penalty relative to the post-Task-A weights). The LLS-4 prototype additionally triggers one sleep cycle after Task B training, synthesizing one adapter from its accumulated memory buffer. All models are then re-evaluated on Task A (*retention*) and on a held-out combined-input set (*zero-shot synthesis*).

Metrics. Parameter count is measured directly via `fvcore`. A pseudo-accuracy score, $\text{acc} = \max(0,\, 100 - 50 \cdot \text{MSE})$, converts regression MSE to a percentage for readability. We do not report inference FLOPs: although the benchmark script attempts an `fvcore` FLOP count, the qualitative labels ("High/High/Low") that appeared in an earlier draft of this table were not derived from that measurement and are withdrawn here as unverified.

5 Results

Table 1 reports the single-run pilot outcome.

Table 1: Pilot benchmark results, synthetic proxy task, single run, unmatched parameter budgets. See Section 6 before interpreting these numbers as comparative evidence.

Metric	LSTM	Custom EWC	LLS-4 (prototype)
Parameter count	4,930	4,610	22,552
Inference FLOPs	not empirically measured in this run		
Task A retention after Task B	33.5%	49.2%	**4.5%**
Zero-shot combined-task score[†]	0.0%	0.0%	71.0%

[†] See Section 6: the combined-task target coincides with the zero vector, which we identify as a likely source of inflation independent of genuine synthesis.

Retention. Contrary to the architecture's design intent, the LLS-4 prototype retained Task A performance *worse* than either baseline. Root-causing this: the CfC backbone and its projection layers are trained directly on Task B data with no protection mechanism analogous to EWC's

penalty or PNN's column-freezing. The sleep-phase adapter is only invoked through the context gate for inputs whose latent query is close to a synthesized key (Equation 6); it was not, in this pilot, positioned to correct the base network's degraded output on the original Task A input distribution. In its current form, the architecture protects *new* adapters from being overwritten, but does not protect the *base network itself* from being overwritten – which is precisely the failure mode the architecture was intended to solve.

Zero-shot synthesis. We report the 71.0% figure for completeness but do not treat it as validated evidence of skill combination, for a specific, checkable reason: the evaluation target for the combined-task input set is $y_{\text{combo}} = \mathbf{0}$, because $\frac{1}{2}(\sin x + (-\sin x)) = 0$ and likewise for the cosine term. Under the pseudo-accuracy metric, *any* model whose output magnitude on these inputs happens to be small – for reasons unrelated to genuinely blending two learned skills – will score well. Since adapter outputs pass through a randomly-initialized 2-layer MLP with no output-scale constraint, a non-trivial baseline rate of small-magnitude outputs is plausible without appeal to synthesis at all. We are not aware of a variant of this experiment in which the zero-shot target is a non-degenerate function of both tasks (Section 7), so we cannot currently distinguish genuine synthesis from this artifact.

6 Limitations

We list every limitation we are aware of, rather than a selected subset, because we intend this table and this prototype to be usable as a starting point for a rigorous follow-up rather than as a finished result:

1. **Retention regression.** LLS-4's measured retention (4.5%) is below both baselines, contradicting the paper's motivating claim; root cause identified above.
2. **Zero-shot metric degeneracy.** The 71.0% figure is confounded with output-magnitude shrinkage toward a target that is, by construction, the zero vector.
3. **Parameter parity violated.** LLS-4 uses 22,552 parameters versus 4,610–4,930 for the baselines (a $\sim$4–5$\times$ difference), so Table 1 is not a controlled comparison in the sense normally required of benchmark studies.
4. **FLOPs unmeasured.** No empirical FLOP count is currently available for any of the three models.
5. **Single run, no seed averaging.** All numbers reflect one random seed; we cannot currently report variance or confidence intervals.
6. **No ablation.** We have not yet run LLS-4 with the sleep phase disabled, which would be required to attribute any future positive result specifically to the synthesis mechanism rather than to the CfC backbone alone.
7. **Non-standard EWC implementation.** Our EWC baseline is a minimal, hand-written diagonal-Fisher implementation with an untuned λ_{EWC}, not the community-standard `Avalanche` library implementation; it may not represent EWC's best achievable performance.
8. **Task–domain mismatch.** The benchmark uses a synthetic sinusoidal regression proxy, not the navigation-and-obstacle-avoidance environment (`environment.py`) that motivates and is described alongside the architecture. No quantitative result in this paper currently reflects the target domain.

7 Future Work

The following concrete steps are, in our view, prerequisites for any subsequent version of this paper to support a comparative claim:

1. Freeze or otherwise protect the CfC backbone's parameters during subsequent-task training (mirroring EWC's penalty or PNN's column isolation), so that skill retention is a property of the base network rather than left entirely to the adapter mechanism.
2. Redesign the zero-shot evaluation task so that the ground-truth combined-task target is not the arithmetic mean of the two source tasks, removing the degeneracy identified in Section 5.
3. Match parameter budgets across all three models before comparing retention or zero-shot scores.
4. Replace the custom EWC implementation with `Avalanche`'s validated implementation, and tune λ_{EWC}.
5. Run all experiments across at least 5 random seeds and report means with confidence intervals.
6. Run the sleep-on/sleep-off ablation.
7. Evaluate directly inside `environment.py` using episodic reward and target-reach rate as the primary metrics, rather than a synthetic regression proxy.
8. Measure inference FLOPs empirically via `fvcore` or `thop` and report the measured values.

8 Conclusion

We have presented LLS-4, an architecture that proposes to decouple fast online policy learning (a CfC liquid network) from offline concept consolidation (latent-space interpolation distilled into lightweight, attention-routed adapters), and a fully working prototype implementing the full pipeline, including an interactive dashboard for inspecting the latent memory manifold during synthesis (Figure 2). A first pilot comparison against an LSTM and a custom EWC baseline does not currently support the architecture's central forgetting-resistance claim: measured retention is below both baselines, and a secondary zero-shot metric is confounded by a target-degeneracy artifact that we disclose in full. We regard the architecture and the open pipeline – rather than the current numbers – as this paper's contribution, and we have set out a specific, itemized experimental protocol (Section 7) that would need to be completed before a comparative claim against EWC or standard replay methods could be responsibly made.

Acknowledgments

The author thanks the open-source maintainers of `ncps`, `PyTorch`, and `fvcore`, on which this prototype depends.

Table 2: Prototype hyperparameters.

Component	Value
CfC hidden size	32
Latent dimension	16
Decoder	$16 \to 32 \to \text{ReLU} \to 6$
Adapter (per synthesized concept)	$4 \to 16 \to \text{ReLU} \to 2$ (< 200 params)
Interpolation coefficient α	0.5
Temporal weight λ (Eq. 4)	0.1
Adapter optimizer	Adam, lr = 0.01
Adapter training	50 epochs / 200 synthetic steps (standalone); 30/100 (interactive)
Memory buffer capacity	10,000

A Implementation Details and Dashboard

The full prototype (`environment.py`, `working_memory.py`, `sleep_engine.py`, `context_gate.py`, `app.py`, `benchmark.py`) is implemented in Python 3.10 / PyTorch (CPU) and totals under 600 lines. Table 2 lists the hyperparameters used throughout.

A Gradio/Plotly dashboard (Figure 2) visualizes the live 2D trajectory and a PCA projection of the 16-dimensional memory manifold in real time, and streams the sleep-phase computation (scoring, interpolation, decoding, adapter training) as a sequence of console log events so that the offline consolidation step is observable rather than opaque. We note this dashboard as an engineering artifact that aids debugging and demonstration; it is not itself evidence for or against the architecture's core claims.

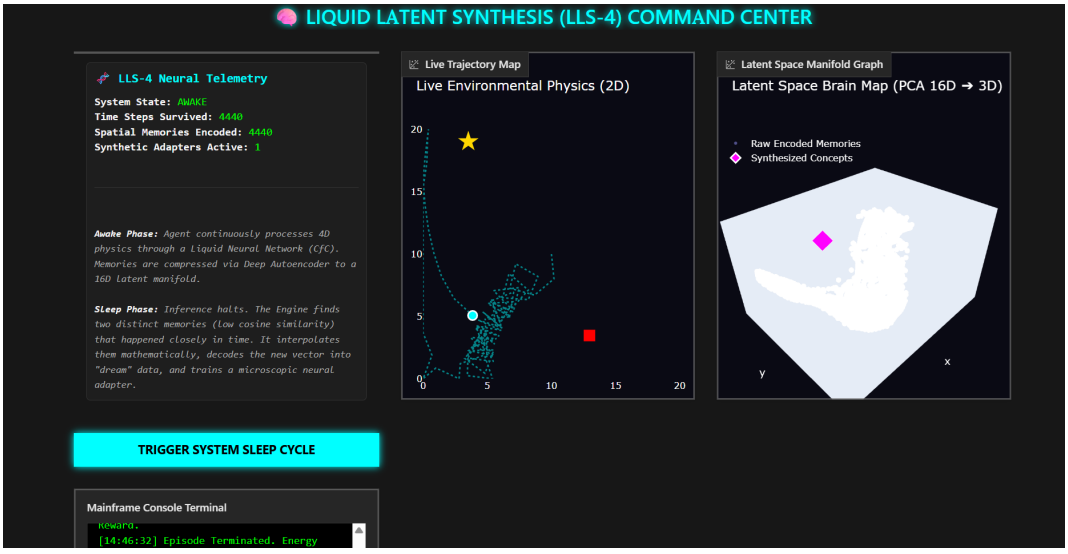

Figure 2: Prototype dashboard during the awake phase. Left: live 2D trajectory (target = gold star, obstacle = red square, agent trail = cyan). Right: PCA projection of the 16-dimensional memory manifold, with one previously synthesized adapter concept shown as a magenta diamond.

References

[1] M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *Psychology of Learning and Motivation*, 24:109–165, 1989.

[2] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[3] F. Huszár. Note on the quadratic penalties in elastic weight consolidation. *Proceedings of the National Academy of Sciences*, 115(11):E2496–E2497, 2018.

[4] F. Zenke, B. Poole, and S. Ganguli. Continual learning through synaptic intelligence. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 3987–3995, 2017.

[5] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.

[6] H. Shin, J. K. Lee, J. Kim, and J. Kim. Continual learning with deep generative replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 2990–2999, 2017.

[7] L. Pellegrini, G. Graffieti, V. Lomonaco, and D. Maltoni. Latent replay for real-time continual learning. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.

[8] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert. iCaRL: Incremental classifier and representation learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[9] M. Lechner, R. Hasani, A. Amini, T. A. Henzinger, D. Rus, and R. Grosu. Neural circuit policies enabling auditable autonomy. *Nature Machine Intelligence*, 2(10):642–652, 2020.

[10] R. Hasani, M. Lechner, A. Amini, L. Liebenwein, A. Ray, M. Tschaikowski, G. Teschl, and D. Rus. Closed-form continuous-time neural networks. *Nature Machine Intelligence*, 4(11):992–1003, 2022.

[11] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.